Analysis and Design of Algorithms

NUS National University of Singapore

Algorithms

CS3230

Tutorial

Week 8

Question 1

Which of the following statements is **false**?

- The amortized cost for insert in dynamic tables is $\Theta(1)$.
- In the accounting method, the amortized cost $\hat{c}_i$ is always greater than the actual cost c_i of an operation.
- $\sum_{i=1}^{n} \hat{c}_i - \sum_{i=1}^{n} c_i \geq 0$ where $\hat{c}_i$ and c_i are the amortized and actual costs of the i-th operation respectively.

Question 1

Answer: B is false

- **A**: For insert in dynamic tables allocate \$3.
  - \$1 used immediately for insert
  - \$2 used later to transfer to new array when array doubled
- **C**: $$\sum_{i=1}^{n} \hat{c}_i - \sum_{i=1}^{n} c_i \geq 0.$$
  - Sum of amortized cost must be larger than sum of total cost (bank balance ≥ 0)
- **B is false**: $\hat{c}_i > c_i$ does not always hold, e.g. insert when table doubled

Question 2

We have a queue-based data structure with the following three operations:

ENQUEUE: Adds an element to the back of the queue.

DEQUEUE: Removes an element from the back of the queue.

CLEAR: Removes elements one at a time from the back of the queue until the queue is empty.

Let the potential function $\Phi(D_i)$ be the number of elements in the queue after operation i where we start with the empty queue D_0. Let the cost of adding and removing each element be 1. What is the amortized cost of ENQUEUE, DEQUEUE, and CLEAR? Note that you cannot dequeue or clear an empty queue.

- ENQUEUE: 0, DEQUEUE: 0, CLEAR: 0
- ENQUEUE: 2, DEQUEUE: 0, CLEAR: 0
- ENQUEUE: 0, DEQUEUE: 2, CLEAR: 0
- ENQUEUE: 1, DEQUEUE: 1, CLEAR: 1

Question 2

Answer: B

Amortized cost = real cost + change in potential.

Let C be current number of queue elements (current potential).

- ENQUEUE: 1 + [(C+1) - C] = 2
- DEQUEUE: 1 + [(C-1) - C] = 0
- CLEAR(k): k + [(C-k) - C] = 0,
  (for removing k elements)

Question 3

Consider a data structure that is based on a queue with $O(1)$ ENQUEUE(a) and DEQUEUE() operations. We construct two new operations ADD(A) and DELETE(k) to create four possible operations on the data structure:

1. ENQUEUE(a) adds the element a to the queue.
2. DEQUEUE() dequeues a single element from the queue.
3. ADD(A) adds the elements in the set A to the queue. The size of A can be as large as required by the user.
4. DELETE(k) dequeues k elements from the queue. The number k is as large as required by the user.

What is the largest set of operations that can be added to the data structure such that all operations are amortised $O(1)$?

- ENQUEUE(a) and DEQUEUE() only.
- ENQUEUE(a), DEQUEUE() and ADD(A) only.
- ENQUEUE(a), DEQUEUE() and DELETE(k) only.

Question 3

Answer: C. ENQUEUE, DEQUEUE, and DELETE(k)

- ADD(A) cannot be amortized $O(1)$.
  - Actual cost is proportional to $|A|$, so bank balance negative if allocate constant amount of amortized cost, but $|A|$ gets large enough.

Need $$\sum_{i=1}^{n} \hat{c}_i - \sum_{i=1}^{n} c_i \geq 0.$$

Question 3

- For ENQUEUE, DEQUEUE, and DELETE(k)
  - Can assign \$2 to ENQUEUE, and \$0 to both DEQUEUE and DELETE(k).
  - From the \$2 for ENQUEUE, can pay \$1 to enqueue the element, and the other \$1 to dequeue each element DEQUEUE and DELETE(k)
- If charge \$2$|A|$ to ADD(A), then bank balance will never be negative with ADD(A) operation added as well.

Question 4

We would like to construct a queue using two stacks. We call the stacks Stack1 and Stack2. The operations are:

```
ENQUEUE:

push onto Stack1.

DEQUEUE:

if Stack2 is empty

    while Stack1 is not empty

        Stack2.push(Stack1.pop())

Stack2.pop()
```

What is the **amortized runtime** and **worst case runtime** per DEQUEUE operation in a sequence of n operations?

- $\Theta(1)$ and $\Theta(1)$
- $\Theta(1)$ and $\Theta(n)$
- $\Theta(n)$ and $\Theta(1)$
- $\Theta(n)$ and $\Theta(n)$

Question 4

Answer: B

- Assign $2 for ENQUEUE
  - $1 to store in Stack1
  - $1 to move to Stack2 when Stack2 empty
- Assign $1 for DEQUEUE for Stack2.pop()
- Amortized cost for both ENQUEUE and DEQUEUE are both $\Theta(1)$
- Worst case for DEQUEUE need to move all elements to Stack2, so $\Theta(n)$